

CLASSES AND OBJECTS

A class is a user defined prototype that defines a set of attributes (data members or variables) and methods that characterises any class object. An object is a unique instance of a class. So far we have been working with objects and these objects belong to Python built-in classes. For better understanding, suppose we create a variable named 'fruit' and we assign to it a string value of "Banana". Then, "Banana" stored in the fruit variable is an object that belongs to the Python String class. With the object, we can access all attributes of the class.

```
In [ ]: fruit = "Banana"
print(f"{fruit} is of {type(fruit)}")
```

We can define our own classes to create objects that carry out a variety of related tasks or represent information in a convenient way. We use the class keyword to create a class.

```
In [1]: class Person():
    pass
```

THE __init__() FUNCTION

All classes have the __init__() function which is also known as the class constructor. The function is always executed when a new instance of the class is created. It is used to assign values to object properties or other operations that are necessary to do when the object is being created.

The first parameter of the __init__() function is self (it can be any other name but it must be the first parameter). Self represents the current instance of the class which is used to access a class attributes.

```
In [2]: class Figure():
    pass

In [3]: def Figure():
    return

In [7]: class Person():
    #initialization
    def __init__(self, name, age, gender):
        self.name = name
        self.age = age
        self.gender = gender

    def describe_person(self):
        print(f"My name is {self.name}. I am a {self.gender}. I am {self.age} years old.")
```

A class can contain methods. In the above code, we created a method called describe_person that prints the name, age and gender of a person. Let's see how to create a class object and access its data members. We use dot notation to access a class data members.

```
In [5]: p1 = Person('Mitchelle', 23, 'female')

In [6]: print(p1.name)
Mitchelle

In [9]: p1 = Person('Damola', 24, 'male')

In [10]: p1.name
Out[10]: 'Damola'

In [11]: p1.describe_person()
My name is Damola. I am a male. I am 24 years old.

In [12]: p2 = Person('Michael', 20, 'male')
print(p2.age)
20

In [55]: p2.describe_person()

-----
NameError                                Traceback (most recent call last)
<ipython-input-5-78bf9c398c30d> in <module>
----> 1 p2.describe_person()
NameError: name 'p2' is not defined
```

p1 and p2 are objects of the class 'Person'. They are unique instances of the class person with which we can access the variables and methods of the class. Another way to use class objects is:

```
In [4]: p3 = Person('Daniel', 18, 'male')
print(Person.describe_person(p3))

-----
TypeError                                Traceback (most recent call last)
<ipython-input-4-ea05db43371a> in <module>
----> 1 p3 = Person('Daniel', 18, 'male')
      2 print(Person.describe_person(p3))
TypeError: Person() takes no arguments
```

In the above code, we accessed the class method by using the class name and passed the object (unique instance) p3 as an argument to the describe_person method. Remember the self parameter represents the current instance of a class and in this case, the instance is p3.

INSTANCE AND CLASS ATTRIBUTES

Instance variables are variables defined in a class for which each instantiated object has its own values (instance). For example, in the Person class, each person object has its own name, age and gender. The self parameter helps us to keep track of individual instances of a class (ie individual values of an object).

Unlike instance variables that are specific to each instance, class variables are the same for all instances. We access class variables with the class name. For example, in the code below, Suppose we want to capture the names and nationality of residents in Nigeria, the variable 'residence' is a class attribute and it is same for all residents. The name and nationality variables are instance variables which varies for each resident instance/object.

Instance methods are defined inside a class and are used to get the contents of an instance. They can also be used to perform operations with the attributes of our objects. Just like the __init__ function, the first parameter of an instance method is the self parameter.

```
In [12]: class Residents:
    #class variable
    residence = 'Nigeria'

    def __init__(self, name, nationality):
        #instance variable
        self.name = name
        self.nationality = nationality

    #instance method
    def describe_person(self):
        print(f"My name is {self.name}. I am a {self.nationality} and I live in {Residents.residence}.")

In [10]: r = Residents("Brian", "British")
r.describe_person()
My name is Brian. I am a British and I live in Nigeria.

In [11]: r2 = Residents("Damola", "Nigerian")
r2.describe_person()
My name is Damola. I am a Nigerian and I live in Nigeria.
```

DUNDER METHODS

Dunder methods are methods with double underscore before and after the method names. They have a special meaning and they are considered as private methods. There are a lot of them in Python. Let's see all the methods for the int class, look out for its dunder methods.

```
In [ ]: print(dir(int))

The __init__, __repr__, __add__, __sub__ etc are dunder methods. The __add__ method allows a programmer to use the + operator with objects. The __repr__ method allows the programmer to tell Python how to print out the object. Let's look at an example of a class with dunder methods.

The first step in creating a class is to define its __init__ and __repr__ methods. For our example, we are going to define a Point class that accepts two arguments, x and y. A Point object would be represented by a string 'Point(x, y)' with appropriate values for x and y. We also want our class to support simple vector operations for points, so we are going to add the __add__ and __sub__ dunder methods. For two points, (x1, y1) + (x2, y2) = (x1+x2, y1+y2). Similarly, (x1, y1) - (x2, y2) = (x1-x2, y1-y2).
```

```
In [14]: class Point():
    def __init__(self, x, y):
        self.x = x
        self.y = y

In [15]: a = Point(5,6)
print(a)
<_main_.Point object at 0x106b84fd0>

In the code above, the print function display an object in memory when we passed a point object as a parameter. To make the print function display/represent the point object nicely, we will need to implement the __repr__ dunder method for the point class.
```

```
In [97]: b = Point(2,3)
a,b
Out[97]: (<_main_.Point at 0x114653590>, <_main_.Point at 0x1150f6c90>)
```

In the above code cell, we got an exception. This is because the + operator has not been supported for the Point object. To make the point class have a + functionality, we must write the __add__ dunder method for the point class. Now, look at the code below.

```
In [21]: class Point():
    def __init__(self, x, y):
        self.x = x
        self.y = y

    #prints out a point object in the form "Point(x, y)"
    def __repr__(self):
        return f"Point(({self.x}, {self.y})"

    #allows the use of + operator with two point objects
    def __add__(self, other):
        if isinstance(other, Point):
            return Point(self.x + other.x, self.y + other.y)
        else:
            raise TypeError ('Expected value should be of Type Point')
    #TO DO: write the __sub__ method

In [22]: a = Point(5,6)
print(a)
Point(5, 6)

We have defined a Point class with the __add__ and __repr__ methods. The __add__ method checks if the argument passed to it is an instance of a Point (ie if it is a point object) because we can only add point objects together. We cannot add an integer or floating point number to a Point. Now, let's interact with the class and its methods.
```

```
In [23]: p1 = Point(3,5)
p2 = Point(3,7)
p1 + p2
Out[23]: Point(6, 12)

In [101]: p1 + 3
-----
TypeError                                Traceback (most recent call last)
<ipython-input-101-6c52a5389135> in <module>
----> 1 p1 + 3

<ipython-input-98-94167103d5e2> in __add__(self, other)
    15         return Point(self.x + other.x, self.y + other.y)
    16     else:
--> 17         raise TypeError ('Expected value should be of Type Point')
    18     #TO DO: write the __sub__ method
TypeError: Expected value should be of Type Point
```

EXERCISE:

Extend the Point class to contain 2 other methods. The __mul__ method to allow us use * operator with point objects and a distance method to calculate the distance between 2 points. The __mul__ method should take care of scalar multiplication and inner product. We can define scalar multiplication given a point P and a scalar a as **aP = a(x, y)=(ax, ay)** and we can define the inner product for points **P1Q as P·Q = (x1,y1)·(x2,y2) = x1x2 + y1y2**. Given 2 points **P1=(x1,y1)** and **P2=(x2,y2)**, the distance, D, between them is denoted as **D=[P2-P1]=√[(x1-x2)²+(y1-y2)²]**.

```
In [102]: class Point():
    def __init__(self, x, y):
        self.x = x
        self.y = y

In [103]: a = Point(5,6)
print(a)
<_main_.Point object at 0x1150e9290>

In [104]: b = Point(2,3)
a,b
Out[104]: (<_main_.Point at 0x1150e9290>, <_main_.Point at 0x114678290>)

In [158]: class Point():
    def __init__(self, x, y):
        self.x = x
        self.y = y

    #prints out a point object in the form "Point(x, y)"
    def __repr__(self):
        return f"Point(({self.x}, {self.y})"

    #allows the use of * operator with two point objects
    def __mul__(self, other):
        if isinstance(other, Point):
            return Point(self.x * other.x, self.y * other.y)
        else:
            raise TypeError ('Expected value should be of Type Point')

In [159]: a = Point(5,6)
print(a)
Point(5, 6)

In [160]: p1 = Point(3,5)
p2 = Point(5,7)
p1 * p2
Out[160]: Point(15, 35)
```

OBJECT ORIENTED PROGRAMMING (OOP)

Now that we understand the concepts of classes and objects, let's dive into the world of OOP. OOP is a programming paradigm that is based on the concept of creating objects. OOP supports code reusability and reduces code repetition. The four OOP concepts are:

1. Encapsulation
2. Inheritance
3. Abstraction
4. Polymorphism

ENCAPSULATION

Encapsulation is a concept of OOP that is used to restrict access to variables and methods (private variables and methods). It is used to prevent data from direct modification. In Python, we denote private variables and methods by prefixing its name with a single or double underscore (_ or __). For greater program security and to prevent unintended data corruption, it is good to make use of private attributes in our program. For example, suppose the salary of every employee in ACME company is calculated as **base salary * (base salary * level/10)**. We would want the base salary to be modified outside our code as this would affect the salary of all employees. For this reason, it is best to define the base salary as a private variable.

```
In [23]: class Employee():
    #private variable
    _base_salary = 50000

    def __init__(self, name, level, position):
        self.name = name
        self.level = level
        self.position = position

    #private method
    def _calculate_salary(self):
        return (Employee._base_salary + (Employee._base_salary * (self.level/10)))

    def generate_paylip(self):
        print(f"Name: {self.name}\nPosition: {self.position}\nLevel: {self.level}\nSalary: {self._calculate_salary}")

    def setBaseSalary(self, new_base_salary):
        Employee._base_salary = new_base_salary

In [24]: a = Employee('Angela', 3, 'Clerk')
a.generate_paylip()
Name: Angela
Position: Clerk
Level: 3
Salary: 65000.0
```

```
In [25]: print(a._base_salary)
50000

In [24]: a._base_salary = 50000
print(a._base_salary)
50000
```

INHERITANCE

Inheritance is a concept of OOP where a class inherits all attributes and methods from another class. The class that inherits is called a child/sub/derived class while the class that is being inherited is called a parent/super/base class. A child class can override or extend the functionality of a parent class. In Python, the most basic type of class is the object class in which all other python classes inherit from.

In Python 3, when you define a new class, it implicitly uses object as the parent class. So the following two class definitions are equivalent:

```
In [ ]: class person(object):
    pass

In [ ]: class person():
    pass
```

To explain the concept of inheritance, we are going to create 3 classes: Person, Employee and Student. Since employees and students are persons, we are going to make them inherit the person class. The common attributes and methods for both employee and students would be contained in the Person class while the different attributes and methods would be contained in their respective classes.

```
In [27]: class Person():
    def __init__(self, name, age, gender, ID):
        self.name = name
        self.age = age
        self.gender = gender
        self.ID = ID

    def ID_Card(self):
        print(f"Identity Number: {self.ID}\nName: {self.name}\nGender: {self.gender}\nAge: {self.age}")

In [32]: class Employee(Person):
    def __init__(self, name, age, gender, ID, company, portfolio, salary):
        #calling the parent constructor with super() function
        super().__init__(name, age, gender, ID)
        #Person.__init__(self, name, age, gender, ID)
        self.company = company
        self.portfolio = portfolio
        self.salary = salary

    def set_salary(self, new_salary):
        self.salary = new_salary

In [ ]: class Student(Person):
    def __init__(self, name, age, gender, ID, school, course, level):
        #calling the parent constructor with the parent name
        Person.__init__(self, name, age, gender, ID)
        self.school = school
        self.course = course
        self.level = level
    def update_level(self):
        self.level +=100
```

Child classes inherit all attributes and methods of its parent class including the __init__() method. When we define an __init__ method within a child class, the child class __init__ function overrides that of its parent class. To keep the inheritance of the parent __init__ method, we call it within the child class __init__ definition using the super function or the name of the parent class as seen above. Now, let's create Employee and Student objects.

```
In [ ]: emp = Employee('Charles', 20, 'male', 'EMP206765', 'ACME', 'Developer', 25000)
emp.ID_Card()

In [ ]: emp.set_salary(75000)
print(emp.salary)

In [ ]: std = Student('Diana', 19, 'female', 'STD23451', 'UI', 'Computer Science', 200)
std.ID_Card()
```

```
In [ ]: std.update_level()
print(std.level)

Child objects are instance of its parent class. The isinstance() function is used to determine if an instance is also an instance of a certain parent class.
```

```
In [ ]: print(isinstance(emp, Person))

In [ ]: print(isinstance(emp, Employee))

In [ ]: print(isinstance(emp, Student))

In [1]: class Customer():
    def __init__(self, name, dob, gender):
        self.name = name
        self.dob = dob
        self.gender = gender

    def details(self):
        print(f"Name: {self.name}\nGender: {self.gender}\ndob: {self.dob}")

In [2]: class Account():
    def __init__(self, name, dob, gender, address, phonenumber):
        #calling the parent constructor with super() function
        super().__init__(name, age, gender, ID)
        #Person.__init__(self, name, dob, gender)
        self.address = address
        self.phonenumber = phonenumber

    def create_account(self, new_account):
        self.account = new_account

In [3]: class Login:
    def __init__(self, name, dob, gender, username, password):
        Customer.__init__(self, name, dob, gender)
        self.username = username
        self.password = password

    def login(self):
        print(f"Username: {self.username}\nPassword: {self.password}")

In [4]: a = Login('Damola', '24', 'male', 'jefef', 'damola8234')
```

```
In [19]: a.login()
Username: jefef
Password: damola8234
```

```
In [13]: class Student:
    def __init__(self, name, age, gender, matric_number):
        self.name = name
        self.age = age
        self.gender = gender
        self.matric_number = matric_number

    def student_info(self):
        print(f"Name: {self.name}\nGender: {self.gender}\nAge: {self.age}\nMatric_number: {self.matric_number}")

In [26]: class Department:
    def __init__(self, name, age, gender, department):
        self.department = department
        student_dept(self, student_dept):
        self.department = student_dept

In [30]: class Course(Student):
    def __init__(self, name, age, gender, matric_number, course):
        Student.__init__(self, name, age, gender, matric_number)
        self.course = course

    def student_course(self, student_Course):
        self.course = student_course

In [44]: class Result(Course):
    def __init__(self, name, age, gender, matric_number, department, course, result):
        Course.__init__(self, name, age, gender, matric_number, course)
        self.department = department

    def student_result(self):
        print(f"Name: {self.name}\nGender: {self.gender}\nAge: {self.age}\nMatric_number: {self.matric_number}")

In [49]: a = Result('Damola', '24', 'Male', '14/2729', 'ces', 'Computer science', 67)

In [50]: a.student_result()
Name: Damola
Gender: Male
Age: 24
Matric number: 14/2729
Course: Computer science
Department: ces
Result: 67
```

ABSTRACTION

Abstraction is an OOP concept that allows a programmer to hide the real implementation and users only know how to use it. In Python, we use abstract classes and interface to achieve abstraction. An abstract class is a class that provides incomplete functionality and contains at least one abstract method. An abstract method that generally has no implementation. Child classes which inherit the abstract class provides implementation for the abstract methods. Concrete methods are methods that have implementation.

Important points about abstract class in Python

1. Abstract class can have both concrete methods as well as abstract methods.
2. Abstract class works as a template for other classes. Using an abstract class you can define a generalized structure without providing complete implementation of every method. Methods which provide common functionality to be used for all defined classes are defined as concrete methods in abstract class where as the methods where implementation may vary for all defined abstract methods.
3. Abstract class can't be instantiated so it is not possible to create objects of an abstract class.
4. Generally abstract methods defined in abstract class don't have any body but it is possible to have abstract methods with implementation in abstract class. Any sub class deriving from such abstract class still needs to provide implementation for such abstract methods.
5. If any abstract method is not implemented by the derived class Python throws an error.

```
In [65]: from abc import ABC, abstractmethod
class Staff(ABC):
    base_pay = 0

    def __init__(self, name, job_title, staff_id):
        self.name = name
        self.job_title = job_title
        self.staff_id = staff_id

    #concrete method
    def ID_Card(self):
        print(f"Staff ID: {self.staff_id}\nName: {self.name}\nPosition: {self.job_title}")

    # Abstractmethod
    def calculate_payment(self):
        pass

As seen above, we imported ABC and abstractmethod from the abc(Abtract Base Class) module. In Python, a class is considered an abstract class if and only if it inherits from the ABC class. Also, in abstract classes, abstract methods must be decorated using the @abstractmethod decorator.
```

In real life examples, a staff could be part-time or full-time. Now, lets create concrete classes that would inherit our abstract staff class and provide implementation for the abstract method.

```
In [66]: class Staff(Staff):
    def __init__(self, name, job_title, staff_id, hours_of_work):
        super().__init__(name, job_title, staff_id)
        self.hours_of_work = hours_of_work

    #Implementation of calculate_salary for Part time staff
    def calculate_payment(self):
        self._base_pay = 500
        amount = self.hours_of_work * self._base_pay
        return "{:.2f}".format(amount)

In [67]: class PT_Staff(Staff):
    def __init__(self, name, job_title, staff_id, level):
        super().__init__(name, job_title, staff_id)
        self.level = level

    #Implementation of calculate_salary for Full time staff
    def calculate_payment(self):
        self._base_pay = 50000
        amount = ((self.level/10)* self._base_pay)*self._base_pay
        return "{:.2f}".format(amount)

In [68]: stf = Staff('Amanda', 'Clerk', 'CL123')

In [69]: pt = PT_Staff('Andy', 'Bricklayer', 'PT10087', 5)
pt.ID_Card()
Staff ID: PT10087
Name: Andy
Position: Bricklayer

In [133]: print(f"Your payment for {pt.hours_of_work} hours of work is {pt.calculate_payment()}")
Your payment for 5 hours of work is 2500.00

In [148]: ft = PT_Staff('Amanda', 'Sales Manager', 'FT43216', 9)
ft.ID_Card()
Staff ID: FT43216
Name: Amanda
Position: Sales Manager

In [149]: print(f"Your salary as a level {ft.level} officer is ${ft.calculate_payment()}")
Your salary as a level 9 officer is $95000.00
```

POLYMORPHISM

Polymorphism simply means having many forms. In programming, it refers to methods having same name but different signatures (ie different set of parameters). At times, the implementation of a parent class method may not fit perfectly for a child class but with the concept of polymorphism, a child class can re-implement the method in its own class. This concept of modifying a parent method in a child class is known as method overriding.

```
In [ ]: class Bank(object):
    def __init__(self, bank_name, establishment_year):
        self.bank_name = bank_name
        self.establishment_year = establishment_year

    def loan_repayment(self, amount, number_of_years, interest_rate):
        repayment = amount + (amount * (interest_rate/100) * number_of_years)
        return "{:.2f}".format(repayment)

In [ ]: class Non_Interest_Bank(Bank):
    def loan_repayment(self, amount, number_of_years, service_charge):
        repayment = amount + (service_charge * number_of_years)
        return "{:.2f}".format(repayment)
```

In the above code, the non_interest class re-implemented the loan_repayment method of the Bank class because non_interest banks do not deal with interests. When we call the loan_repayment method with a non_interest_bank object, the loan_repayment method of the non_interest_bank class is executed. Similarly, when we call the *loan repayment method of a Bank class*, the *loan repayment method of the Bank class* is executed.

```
In [ ]: UBA = Bank('UBA', 1906)
Jaiz = Non_Interest_Bank('Jaiz', 2000)
print(f"UBA loan_repayment(500000, 5, 4.2)")

In [ ]: print(f"Jaiz loan_repayment(500000, 5, 3000)")

In [4]: from abc import ABC, abstractmethod
class Staff(ABC):
    base_pay = 0

    def __init__(self, name, job_title, staff_id):
        self.name = name
        self.job_title = job_title
        self.staff_id = staff_id

    #concrete method
    def ID_Card(self):
        print(f"Staff ID: {self.staff_id}\nName: {self.name}\nPosition: {self.job_title}")

    # Abstractmethod
    def monthly_payment(self):
        pass

    def yearly_salary(self):
        pass
```

```
In [5]: class Academic_Staff(Staff):
    def __init__(self, name, job_title, staff_id, hours_of_work):
        Staff.__init__(self, name, job_title, staff_id)
        self.hours_of_work = hours_of_work

    #Implementation of calculate_salary for Academic staff
    def monthly_payment(self):
        self._base_pay = 50000
        amount = self.hours_of_work * self._base_pay
        return "{:.2f}".format(amount)

    def yearly_salary(self):
        self._base_pay = 50000
        amount = (self._base_pay)*self._base_pay
        return "{:.2f}".format(amount*12)
```

```
In [6]: class Non_academic_Staff(Staff):
    def __init__(self, name, job_title, staff_id, level):
        Staff.__init__(self, name, job_title, staff_id)
        self.level = level

    #Implementation of calculate_salary for Non-Academic staff
    def monthly_payment(self):
        self._base_pay = 500
        amount = ((self.level/10)* self._base_pay)*self._base_pay
        return "{:.2f}".format(amount*12)

In [7]: a = Academic_Staff('Damola', 'Director', 'AS10087', 5)
a.ID_Card()
Staff ID: AS10087
Name: Damola
Position: Director
```

```
In [6]: print(f"Your Monthly payment for {a.hours_of_work} hours of work is {a.monthly_payment()}")
Your Monthly payment for 5 hours of work is 250000.00
```

```
In [9]: print(f"Your yearly salary for {a.hours_of_work} hours of work is {a.yearly_salary()}")
Your yearly salary for 5 hours of work is 1200000.00
```

```
In [10]: na = Non_academic_Staff('Mimi', 'Teacher', 'NS10067', 13)
na.ID_Card()
Staff ID: NS10067
Name: Mimi
Position: Teacher
```

```
In [11]: print(f"Your monthly payment as a level {na.level} officer is {na.monthly_payment()}")
Your Monthly payment as a level 13 officer is 1150.00
```

```
In [12]: print(f"Your yearly salary as a level {na.level} officer is {na.yearly_salary()}")
Your yearly salary as a level 13 officer is 13800.00
```

```
In [ ]: 
```